Juan D'Elia 930/22 N orden: 18 Ap

Arquitectura y Organización de Computadoras - DC - UBA

Segundo Parcial

Segundo Cuatrimestre 2024

Normas generales

- El parcial es INDIVIDUAL
- Puede disponer de la bibliografía de la materia y acceder al repositorio de código del taller de system programming, desarrollado durante la cursada
- Las resoluciones que incluyan código, pueden usar assembly o C. No es necesario que el código compile correctamente, pero debe tener un nivel de detalle adecuado para lo pedido por el ejercicio.
- Numere las hojas entregadas. Complete en la primera hoja la cantidad de hojas entregadas
- Entregue esta hoja junto al examen. La misma no se incluye en el total de hojas entregadas.

Régimen de Aprobación

- Para aprobar el examen es necesario obtener cómo mínimo **60 puntos**.
- Para promocionar es condición suficiente y necesaria obtener como mínimo **80 puntos** tanto en este examen como en el primer parcial

NOTA: Lea el enunciado del parcial hasta el final, antes de comenzar a resolverlo.

Contexto

Nos encargaron el desarrollo de un kernel para la nueva consola Orga Génesis que cuenta con un procesador x86. Para ahorrar trabajo vamos a tomar nuestro kernel y expandirlo para permitir la ejecución de juegos mediante cartuchos, que contendrán el código y los recursos gráficos (sprites de personajes, fondos). Se incorpora entonces un lector de cartuchos que, entre otras cosas, copiará los gráficos del cartucho a un buffer de video de 4 kB en memoria. Cada vez que el buffer esté lleno y listo para su procesamiento, el lector se lo informará al kernel mediante una interrupción externa mapeada al IRQ 40 de nuestro x86.

Primer ejercicio

Distintas tareas se ocuparán de mostrar los gráficos en pantalla y de realizar otros pre/postprocesamientos de imagen on-the-fly.

Para esto, el sistema debe soportar que las tareas puedan acceder al buffer de video a través de los siguientes mecanismos:

- DMA (Direct Memory Access) - se mappea la dirección virtual `0xBABAB000` del espacio de direcciones de la tarea directamente al buffer de video.
- Por copia - se realiza una copia del buffer en una página física específica y se mapea en la dirección virtual `0xBABAB000`. Cada tarea debe tener una copia única.

El buffer de video se encuentra en la dirección física de memoria `0xF151C000` y **solo debe ser modificable por el lector de cartuchos.**

a) Programar una función `void buffer_dma(pd_entry_t* pd)` que dado el page directory de una tarea realice el mapeo del buffer en modo DMA.

b) Programar una función `void buffer_copy(pd_entry_t* pd, paddr_t phys)` que dado el page directory de una tarea realice la copia del buffer a la dirección física pasada por parámetro y realice el mapeo correspondiente.

Segundo ejercicio

Las tareas configuran en memoria una variable que sirve para comunicar al sistema de qué manera van a acceder al buffer de video.

Cada tarea guarda en la dirección virtual `0xACCE50` (mapeada como r/w para la tarea) un `uint8_t acceso` con posibles valores 0, 1 y 2. El valor 0 indica que la tarea no accederá al buffer de video, 1 que accederá mediante DMA y 2 que accederá por copia. De acceder por copia, la dirección virtual donde realizar la copia estará dada por el valor del registro `ecx` al momento de llamar a `opendevice`.

Cada vez que se indique como completo el buffer, se deberá mappear el mismo a las tareas que **utilicen DMA** y hayan **solicitado acceso** al buffer, o actualizar la copia del buffer de las tareas que **acceden por copia** y hayan **solicitado acceso** al buffer.

Es deseable que cada tarea que accede por copia mantenga una única copia del buffer para no ocupar la memoria innecesariamente.

Para solicitar acceso al buffer de memoria, las tareas deberán informar que desean acceder a él mediante una syscall `opendevice`. El sistema no debe retomar la ejecución de estas tareas hasta que se detecte que el buffer está listo y se haya realizado el mapeo DMA o la copia correspondiente. Una vez que la tarea termine de utilizar el buffer, deberá indicarlo mediante la syscall `closedevice`. Mientras esté en uso el buffer, la tarea deberá poder visualizar su valor actualizado, ya sea por DMA o por copia.

Se muestra el comportamiento esperado en el siguiente esquema, donde las flechas indican que la información actualizada del buffer se disponibiliza para la tarea señalada.

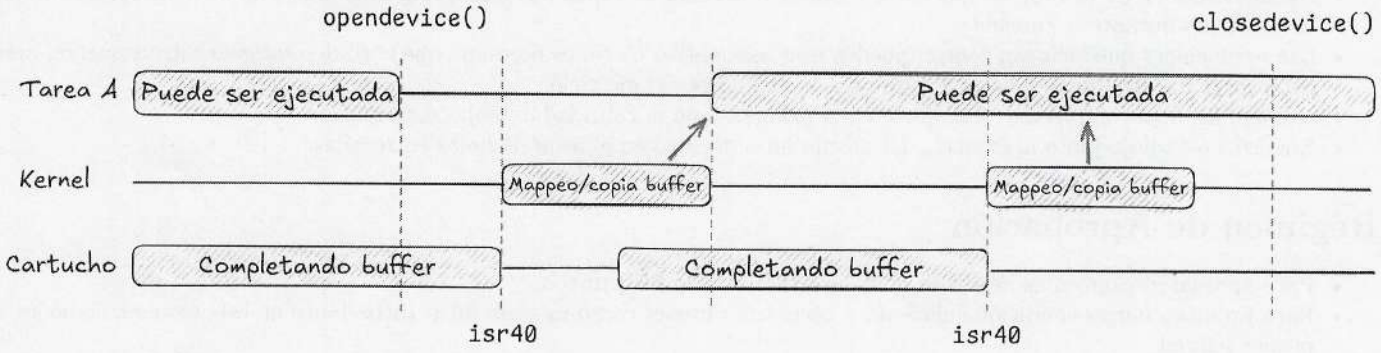


Figure 1: Diagrama schedule tarea, cartucho y kernel

Utilizando las funciones del primer ejercicio, se pide:

a) Programar la rutina que atenderá la interrupción que el lector de cartuchos generará al terminar de llenar el buffer.

Consejo: programar una función `deviceready` y llamarla desde esta rutina.

b) Programar las syscalls `opendevice` y `closedevice`.

A tener en cuenta para la entrega (para todos los ejercicios):

- Indicar **todas las estructuras de sistema** que deben ser modificadas para implementar las soluciones.
- Está permitido utilizar las funciones desarrolladas en los talleres.
- Es necesario que se incluya **una explicación con sus palabras** de la idea general de las soluciones.
- Es necesario escribir todas las asunciones que haga sobre el sistema.
- Es necesaria la entrega de código que implemente las soluciones.

Valeria Juan D'Elía 930122

Ejercicio 1

a)

Para hacer el mapeo puedo usar la función mmu_map_page, que si le paso como parámetro el page directory entry por su formato (que tiene en los 20 bits más altos la dirección de pd_entry) la macro CR3_TO_PAGE_PH va a seguir funcionando. Entonces puedo usar mmu_map_page. (en el taller me anda)

Voy a mapear la direc virtual indicada por DMA a la indicada del buffer de video. Este debe ser únicamente modificado por el buffer así que no le doy permiso de escritura.

```
void buffer_dma(pd_entry_t * pd){
    mmu_map_page((uint32_t) pd, 0xBABA0000, 0xFISICO000, MMU_U|MMU_P);
}
```

MMU_P

Sin write!!

Para buffer copy podemos buscar la dirección física de la virt que nos pasaron por parámetro y usando copy_page copie lo que hay en la direc virtual a la física.

```
void buffer_copy(pd_entry_t* pd, paddr_t phys, vaddr_t virt)
    paddr_t src_phys = virt_to_phys(virt, pd);
    copy_page(phys, src_phys);
```

6 a atrás

6) Primero voy a copiar el buffer del buffer del parámetro a la dirección física indicada en

Hay que copiar el contenido del buffer a la dirección física indicada. Al ser ambas direcciones físicas puedo hacer copy_page.
Por último para mapear ya tengo el page directory y las direcciones así que puedo usar map.

```
void buffer_copy(pd_entry_t* pd, paddr phys, vaddr virt){
                 (pdt)                  (dirección física buffer)
    copy_page(phys, 0xF151C000);
                    (uint32_t)
    mmu_map_page(pd, virt, phys, MMU_U | MMU_P);
}
```

Si hubiera cualquier problema con pasarle el pd a map page (en mi taller funciona) se puede cambiar map page (o hacer otra versión) que tome un pd y en lugar de al principio calcularlo con CR3_TO_PAGE_DIR, usar el directorio pasado por param.
Cabe aclarar que es necesario un único copy_page y un único map (en ambas) ya que se copian los gráficos a 4 KB de memoria (una pág)

Juan DÉlia
93022

Ejercicio 2

a) Idea: Voy a tener marcadas las tareas que hicieron ofrdevice, para que una vez el buffer marque listo, todas aquellas que pidieron acceso se les sea concedido y actualizado. También voy a tener que tener marcadas a las que hicieron closed para sacarselo.
Al no desmarcar el pedido de acceso, ameno que haga un close, cada vez que el buffer interrumpa va a seguir actualizando a las que debe.
Para esta implementación voy a hacer de cuenta (lo voy a hacer después) que tengo una lista que tiene a las tareas ~~que solicitaron acceso al buffer~~. La lista, parecida a la del scheduler va a tener su selector para poder acceder a la TSS de la tarea y buscar su cr3 y ecx para realizar el mapeo y state que indica el RIB de acceso.
Si state es 0 me tengo que asegurar de no actualizar más el buffer para esa tarea (closed marca con 0)

Además las tareas cuando hagan ofrdevice se van a marcar en el scheduler como "processing" y termine el mapeo. Cada vez que ~~vuelva~~ el buffer se complete va a marcar las processing de nuevo como RUNNABLE en su state. Esto para evitar que se ejecuten tareas que pidieron buffer que todavía no se mapeo.

También van a tener otro atributo. PhysGuest, que va a indicar en q' pág física esta la copia del buffer y en virt a donde mapear.

IDT_ENTRY ? (20)

```
isr40:
    pushad
    call pic_finish1
    call device_ready
    popad
    iret
```

asumo q' es de hardware ✓

```
//Cada vez q' termine el buffer actualizo a las tareas en mi nueva lista b_tasks
void device_ready():
    for(int i=0; i<MAX_TASKS; i++){
    uint32_t cr3 = getCR3(b_tasks[i].selector);
    if(b_tasks[i].state==0){ //caso unmap copia y DMA
        Unmap(cr3, 0xBABAB000); virt
        Unmap(cr3, b_tasks[i].phys); //si no está definido es 0 virt
    }
    if(b_tasks[i].state==1){ //le saco la copia y mapeo DMA
        buffer_dma((CR3_TO_PAGE_DIR(cr3));
        Unmap(cr3, virt);
    }
    if(b_tasks[i].state==2){ //le saco el DMA y actualizo/mapeo copia
        //caso me hizo recién ofer, direc virt en ecx
        if(b_tasks[i].virt==0){ //no tengo Página virt p' actualizar
            virt = get_ecx(b_tasks[i].selector); //está en la pila lo último q' hizo
            Unmap(cr3, 0xBABAB000); //DMA          //fue ofer
            Proxima = next_free_user_page();
            buffer_copy(CR3_TO_PAGE_DIR(cr3), Proxima, virt);
        else{ //sino es 0 es xq tengo q' actualizar la copia
            Unmap(cr3, b_tasks[i].virt); //unmapeo la vieja
            Proxima = next_free_user_page();
            b_tasks[i].phys = Proxima
            buffer_copy(CR3_TO_PAGE_DIR(cr3), Proxima, b_tasks[i].virt);
    } for(i=0; i<MAX_TASKS; i++){ //las q' estaban esperando las habilito
        if(sched_tasks[i].state == Processing){
            Sched_enable_task(i);}
    }
}
```

(*) //guardo la virt y phys para que sea la misma

```
b_tasks[i].virt = virt
b_tasks[i].phys = Proxima
```

Para siempre son mismas dir

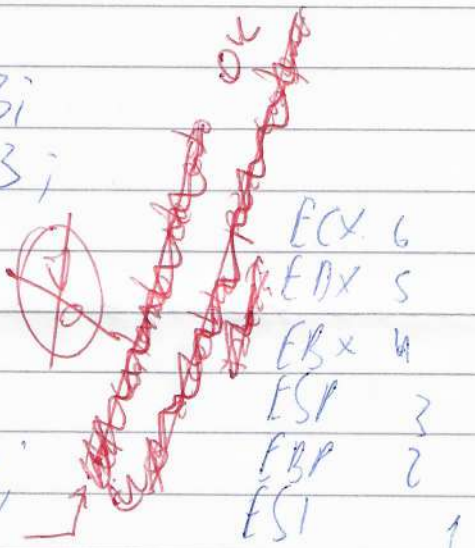
guardo → guardo

Sigue 2a	Juan D'Elia 930/22	

Básicamente la función hace todos mal/unmap necesarios para cada tarea defendiendo de su state (sin importar hace cuanto hayan llamado a bien).
Para esto tengo una lista con structs que (en el mismo orden que el scheduler) tienen c/ tarea con su selector, state, phys y virt que guardan donde está mapeada la copia. Y al final vuelve a habilitar todas las tareas que hicieron o bien (las voy a marcar como Processing en o bien y dos cd también modificando su state)

```
uint16_t
get_cr3(selector) {
    uint16_t idx = selector >> 3;
    return gdt[idx].base->cr3;
}

get_ecx(selector) {
    uint16_t idx = selector >> 3;
    tss_t* tss = gdt[idx].base;
    uint32_t* pila = tss->esp;
    uint32_t ecx = pila[6];
    return ecx;
}
```



La busco de la pila x q' al momento de llamar o bien ecx es válido, después de eso en la interrupción de o bien cambie de contexto y llame a sched_next, así que los regs pueden estar mal, pero en la pila estaban bien, así que la pila de la tss vale

Importante: b_tasks se inicializa con todas las tareas con virt y phys en 0 y state tmb en 0.

Siempre asumí segmentación flat, paginación activada, y que las tareas ya habían corrido al menos una vez.

	Juan D'Elia	
	930/12	

2 b) open device y close device
agrego en la idt ints de nivel 3

```
int_init() {
    IDTENTRY_3(99);   open
    IDTENTRY_3(100);  close
    ...
}
```

Mentira la busco en la tss en la int del buffer

En open voy a tener (si es necesario) q' actualizar la direc virt de la página copia y el state de la tarea en b_tasks. y marcarla (a la open) como Procesando a el scheduler defino en isr99 la SyS call. Además después de un open tengo que hacer jmp far a una tarea q' no esté esperando al buffer.

```
isr99:
    pushad
    mov byte ax, [0x4CC50]; busco el state (1 o 4)
    push ax                ; Si llego 0 va a desmalcar todo
    call actualizar_estado
    add esp, 4
    popad
    iret
```

esto acá no quiero volver a la tarea hasta q' termine de procesar me voy como con el scheduler (isr32)

```
    call sched_next_task
    mov word [sched_task_selector], ax
    jmp far [sched_task_offset]
```

Salto sin importar que me quiero ir, de última idle

```
isr100:
    pushad
    call close
    popad
    iret
```

```
void actualizar_estado(int estado){
    b_tasks[current_task].State = estado;
    sched_tasks[current_task].State = Processing;
}
void close(){ b_tasks[current_task] = 0; } 0 desmalca todo en isr99
```